

Enterprise IT Service Desk

Frontend Technical Documentation

React Application Architecture & Developer Reference

Date: September 2026

Version: 1.0

Package: it-service-desk-ui

Framework: React 18.3.1 + Vite 5.4.8

Table of Contents

1 Introduction & Purpose.....	3
2 Technology Stack.....	3
3 Project Structure.....	4
4 Configuration & Build Setup.....	5
5 Application Entry Points.....	6
6 The API Client Layer.....	7
7 Shared Components — NavBar.....	9
8 Pages.....	9
8.1 UsersPage.....	9
8.2 CategoriesPage.....	10
8.3 TicketsPage.....	10
8.4 TicketDetailPage.....	11
8.5 AuditLogsPage.....	13
9 Application Flow & User Journey.....	13
10 Data Models.....	14
11 State Management Architecture.....	14
12 Backend Integration.....	15
13 Limitations & Future Improvements.....	16
14 Running & Developing the Application.....	17

1. Introduction & Purpose

The Enterprise IT Service Desk is a full-stack web application designed to streamline the management of internal IT support requests within an organisation. The frontend component — documented here — is a single-page application (SPA) built with React 18 and served by the Vite development toolchain. It provides a browser-based interface that communicates exclusively with a FastAPI backend over a RESTful HTTP API.

The application enables IT teams and end-users to raise and track support tickets, manage the people and categories associated with those tickets, and review a complete immutable audit trail of all system activity. The frontend is intentionally lightweight: it contains no authentication layer, no server-side rendering, and no complex state management framework. Every interaction is a direct CRUD operation against the backend REST API, making the codebase straightforward to understand, extend, and maintain.

1.1 Scope of This Document

This document provides a comprehensive technical reference for the React frontend codebase located at `C:\Rakesh\Frontend\frontend\`. It covers every source file, the build configuration, the HTTP API client layer, all five page components, data models, application flow, and guidance for running the application locally. It is intended for developers onboarding to the project, technical reviewers, and anyone seeking to extend or integrate the frontend.

1.2 High-Level Architecture

The frontend is a pure client-side SPA. On first load, the browser downloads the JavaScript bundle from Vite's dev server (or a static file host in production). React Router then handles all subsequent navigation entirely in the browser without any server round-trips for HTML. All data is fetched from the FastAPI backend at `http://localhost:8000` via a transparent Vite proxy that rewrites `/api/*` to `/*`.

2. Technology Stack

The technology choices for this project prioritise simplicity, stability, and minimal dependencies. Every library listed below was a deliberate decision to keep the codebase approachable and free of unnecessary abstraction layers.

Layer	Technology	Version	Notes
UI Framework	React	18.3.1	Functional components + hooks throughout
Routing	react-router-dom	6.26.2	BrowserRouter, Routes, NavLink, useParams, useNavigate
Styling	Bootstrap (CSS only)	5.3.3	Imported as a CSS file; no JS components
Build Tool	Vite	5.4.8	Dev server with HMR + production bundler
React Plugin	@vitejs/plugin-react	4.3.1	Babel-based JSX transform for Vite
Language	JavaScript (JSX)	ES Modules	No TypeScript; .jsx file extension
HTTP Client	Native fetch API	Browser	No Axios or other HTTP library
State Mgmt	useState (React)	—	No Redux, Zustand, or Context API
Testing	None	—	No Jest, Vitest, or RTL configured

2.1 React 18

React 18 is used in its default concurrent-features mode via ReactDOM.createRoot. The project uses exclusively functional components and the built-in useState and useEffect hooks. No class components are present anywhere in the codebase. React.StrictMode is enabled in development, which double-invokes certain lifecycle methods to help surface side-effect bugs early.

2.2 React Router DOM v6

React Router v6 provides declarative client-side routing. The application uses BrowserRouter (HTML5 history API), the Routes/Route components for URL matching, NavLink for the navigation bar with automatic active-link detection, useParams to extract dynamic URL segments (e.g., ticketId), and useNavigate for programmatic redirects (e.g., navigating back to /tickets after a failed ticket fetch).

2.3 Bootstrap 5.3.3

Bootstrap is imported solely as a CSS stylesheet — no Bootstrap JavaScript bundle or Popper.js is loaded. The project uses Bootstrap's utility classes (d-flex, mt-4, col-md-6, etc.), component classes (card, btn, table, badge, navbar, form-control), and contextual colour variants (primary, secondary, info, warning, success, dark) to style the UI. This approach avoids any conflict between Bootstrap's JS and React's DOM management.

2.4 Vite 5

Vite serves as both the development server (with native ESM Hot Module Replacement) and the production bundler (using Rollup under the hood). The dev server proxy configuration is the only non-default Vite setting used by this project: all `/api/*` requests are silently forwarded to the FastAPI backend, eliminating CORS issues during local development.

3. Project Structure

The repository root at C:\Rakesh\Frontend\ contains documentation files and the frontend\ subdirectory. All application source code lives under frontend\src\. The structure is intentionally flat: there are no feature folders, no lazy-loaded chunks, and no nested component hierarchies beyond the single shared NavBar.

```
C:\Rakesh\Frontend\  
├─ HOW-TO-RUN.md           # Setup & run instructions  
├─ Enterprise IT Service Desk.md # Project overview documentation  
└─ frontend\  
    ├─ index.html           # HTML shell - single <div id='root'>  
    ├─ package.json         # Dependencies, scripts, ESM type  
    ├─ vite.config.js       # Build tool & proxy configuration  
    └─ src\  
        ├─ main.jsx         # React DOM entry point  
        ├─ App.jsx          # BrowserRouter + route table  
        └─ api\  
            └─ client.js     # All 20 HTTP API functions  
        ├─ components\  
            └─ NavBar.jsx    # Top navigation bar  
        └─ pages\  
            ├─ UsersPage.jsx  # User CRUD  
            ├─ CategoriesPage.jsx # Category CRUD  
            ├─ TicketsPage.jsx  # Ticket list, filter, create, delete  
            ├─ TicketDetailPage.jsx # Ticket detail & all sub-resources  
            └─ AuditLogsPage.jsx # Read-only system audit log
```

3.1 File Roles Summary

File	Role	Key Imports
index.html	HTML shell; Vite entry point	—
main.jsx	Mounts React app into DOM	React, ReactDOM, App, Bootstrap CSS
App.jsx	Root layout + route definitions	BrowserRouter, Routes, NavBar, all Pages
api/client.js	Centralised HTTP client; 20 exported functions	Native fetch
components/NavBar.jsx	Persistent top navigation	NavLink (react-router-dom)
pages/UsersPage.jsx	Create & delete users	getUsers, createUser, deleteUser
pages/CategoriesPage.jsx	Create & delete categories	getCategories, createCategory, deleteCategory
pages/TicketsPage.jsx	List, filter, create & delete tickets	getTickets, getUsers, getCategories, createTicket, deleteTicket, useNavigate
pages/TicketDetailPage.jsx	Full ticket detail hub	10 API functions, useParams, useNavigate

pages/AuditLogsPage.jsx	Read-only audit log viewer	getAllAuditLogs
-------------------------	----------------------------	-----------------

4. Configuration & Build Setup

4.1 package.json

The package.json declares the project as an ES Module package ("type": "module"), which means all .js and .jsx files in the project are treated as ESM by default — import/export syntax is used everywhere, and CommonJS require() calls are not present.

Scripts

Script	Command	Purpose
dev	vite	Start Vite dev server on port 5173 with HMR
build	vite build	Bundle for production into frontend/dist/
preview	vite preview	Serve the production build locally for verification

Runtime Dependencies

Package	Version	Purpose
bootstrap	^5.3.3	CSS utility/component framework
react	^18.3.1	Core UI library
react-dom	^18.3.1	React DOM renderer
react-router-dom	^6.26.2	Client-side SPA routing

Development Dependencies

Package	Version	Purpose
@vitejs/plugin-react	^4.3.1	Babel-powered JSX transform + Fast Refresh for Vite
vite	^5.4.8	Build tool and development server

4.2 vite.config.js

The Vite configuration file is minimal but includes the most critical piece of development infrastructure: the API proxy. This proxy allows the frontend to call /api/users and have Vite transparently forward the request to the FastAPI backend at http://localhost:8000/users — with the /api prefix stripped.

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
```

```
      changeOrigin: true,  
      rewrite: (path) => path.replace(/^\/api/, '')  
    }  
  }  
}  
}))
```

The rewrite function strips the `/api` prefix before the request reaches FastAPI. This means the frontend code always uses `/api/*` as the base, the FastAPI routes are defined without any `/api` prefix, and no CORS configuration is needed during development because both the frontend and (from the browser's perspective) the API share the same origin (`http://localhost:5173`).

4.3 index.html

The HTML file is the minimal shell required by Vite. It contains a single `<div id="root">` element that React mounts into, and a `<script type="module">` tag pointing to `src/main.jsx`. Vite processes this file, injects the built assets, and serves it as the SPA's sole HTML document. The browser title is set to 'IT Service Desk'.

5. Application Entry Points

5.1 main.jsx — React DOM Bootstrap

main.jsx is the first JavaScript file executed by the browser. Its responsibilities are strictly limited to: importing Bootstrap's CSS globally (so it is available to all components), and mounting the root React component tree into the DOM.

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import 'bootstrap/dist/css/bootstrap.min.css'
import App from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

Using ReactDOM.createRoot (the React 18 API) enables concurrent rendering features. React.StrictMode wraps the entire application, which in development causes React to mount components twice to detect unexpected side effects. There are no Context providers, Redux stores, or other wrappers at this level — the component tree is completely flat.

5.2 App.jsx — Root Component & Route Table

App.jsx defines the top-level layout and the complete client-side route table. Every page in the application is reachable from this file. The layout is simple: the NavBar always renders at the top of the page, followed by a Bootstrap .container with margin-top (mt-4) that houses the current page's content.

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
import NavBar from './components/NavBar.jsx'
import UsersPage from './pages/UsersPage.jsx'
import CategoriesPage from './pages/CategoriesPage.jsx'
import TicketsPage from './pages/TicketsPage.jsx'
import TicketDetailPage from './pages/TicketDetailPage.jsx'
import AuditLogsPage from './pages/AuditLogsPage.jsx'

export default function App() {
  return (
    <BrowserRouter>
      <NavBar />
      <div className='container mt-4'>
        <Routes>
          <Route path='/'
            element={<Navigate
to='/tickets' />} />
          <Route path='/users'
            element={<UsersPage />} />
          <Route path='/categories'
            element={<CategoriesPage />} />
          <Route path='/tickets'
            element={<TicketsPage />} />
        </Routes>
      </div>
    </BrowserRouter>
  )
}
```

```

        <Route path='/tickets/:ticketId' element={<TicketDetailPage />}
    />
        <Route path='/audit-logs' element={<AuditLogsPage />} />
    </Routes>
</div>
</BrowserRouter>
)
}

```

Route Table

Path	Component	Description
/	Navigate to /tickets	Default redirect — root URL always lands on ticket list
/users	UsersPage	Create and delete system users
/categories	CategoriesPage	Create and delete ticket categories
/tickets	TicketsPage	Filterable ticket list with create and delete
/tickets/:ticketId	TicketDetailPage	Full detail view for a single ticket
/audit-logs	AuditLogsPage	Read-only system-wide audit trail

6. The API Client Layer — src/api/client.js

All HTTP communication between the frontend and the FastAPI backend is centralised in a single file: `src/api/client.js`. No page or component calls `fetch()` directly. This design keeps API logic out of UI components, makes it trivial to swap the HTTP transport (e.g., replace `fetch` with `Axios`), and provides a single place to add headers, logging, or error handling globally.

6.1 Base URL & Core Request Function

The base URL is hardcoded as the string `'/api'`. During development, Vite's proxy transparently rewrites this to `http://localhost:8000`. In production, an equivalent reverse-proxy rule would need to be configured on the web server or CDN.

```
const BASE_URL = '/api'

async function request(method, path, body) {
  const options = {
    method,
    headers: { 'Content-Type': 'application/json' },
  }
  if (body !== undefined) {
    options.body = JSON.stringify(body)
  }
  const res = await fetch(`${BASE_URL}${path}`, options)
  if (res.status === 204) return null // DELETE responses with no body
  const data = await res.json()
  if (!res.ok) throw new Error(data.detail)
  return data
}
```

Key behaviours of the `request()` function: `Content-Type` is always set to `application/json` regardless of method. The request body is JSON-serialised only when a body argument is provided (undefined body means no body is sent — correct for GET and DELETE calls). A 204 No Content response (the typical FastAPI response for successful DELETE operations) returns null without attempting to parse an empty body. For any non-2xx response, the function throws an `Error` whose message is taken from `data.detail` — the FastAPI error response field. All callers catch this error and display it to the user.

6.2 User API Functions

```
export const getUsers = () => request('GET', '/users')
export const createUser = (body) => request('POST', '/users', body)
export const deleteUser = (id) => request('DELETE', `/users/${id}`)
```

Function	HTTP	Endpoint	Payload / Notes
<code>getUsers()</code>	GET	<code>/users</code>	Returns array of all user objects
<code>createUser(body)</code>	POST	<code>/users</code>	Body: { name, email, password, role }
<code>deleteUser(id)</code>	DELETE	<code>/users/{id}</code>	Returns null (204). id is a UUID string

6.3 Category API Functions

```
export const getCategories = () => request('GET', '/categories')
export const createCategory = (body) => request('POST', '/categories', body)
export const deleteCategory = (id) => request('DELETE', '/categories/${id}`)
```

6.4 Ticket API Functions

```
export const getTickets = (params='') => request('GET', `/tickets${params}`)
export const getTicket = (id) => request('GET', `/tickets/${id}`)
export const createTicket = (body) => request('POST', '/tickets', body)
export const deleteTicket = (id) => request('DELETE', `/tickets/${id}`)
export const assignTicket = (id, body) => request('PATCH', `/tickets/${id}/assign`, body)
export const updateTicketStatus = (id, body) => request('PATCH', `/tickets/${id}/status`, body)
```

The `params` argument to `getTickets()` is a raw query string such as `'?status=new'`. When no filter is active, it defaults to an empty string. The PATCH endpoints (assign and status) use HTTP PATCH rather than PUT because they update only a subset of the ticket fields. The frontend never uses the backend's PUT endpoints — there are no edit forms for ticket metadata.

6.5 Comment API Functions

```
export const getComments = (ticketId) => request('GET', `/tickets/${ticketId}/comments`)
export const createComment = (ticketId, body) => request('POST', `/tickets/${ticketId}/comments`, body)
export const deleteComment = (ticketId, commentId) => request('DELETE', `/tickets/${ticketId}/comments/${commentId}`)
```

6.6 Attachment API Functions

```
export const getAttachments = (ticketId) => request('GET', `/tickets/${ticketId}/attachments`)
export const createAttachment = (ticketId, body) => request('POST', `/tickets/${ticketId}/attachments`, body)
export const deleteAttachment = (ticketId, attachmentId) => request('DELETE', `/tickets/${ticketId}/attachments/${attachmentId}`)
```

6.7 Audit Log API Functions

```
export const getAllAuditLogs = () => request('GET', '/audit-logs')
export const getTicketAuditLogs = (ticketId) => request('GET', `/tickets/${ticketId}/audit-logs`)
```

`getAllAuditLogs()` fetches every audit record across all tickets. It is used exclusively by `AuditLogsPage`. `getTicketAuditLogs()` fetches only the audit records for a specific ticket and is used in the audit-log card within `TicketDetailPage`. The backend returns records ordered newest-first.

7. Shared Components — NavBar.jsx

The application has exactly one shared component: the NavBar. It is rendered by App.jsx on every route, providing persistent top-level navigation.

The NavBar uses Bootstrap's navbar-dark and bg-dark classes for a dark background with light text. The brand name 'IT Service Desk' is displayed on the left. Four navigation links — Tickets, Users, Categories, and Audit Logs — are rendered using React Router's <NavLink> component, which automatically applies an 'active' class when the current URL matches the link's to prop. The active link additionally receives the fw-semibold Bootstrap class for a visual emphasis that highlights the current section.

```
import { NavLink } from 'react-router-dom'

export default function NavBar() {
  const linkClass = ({ isActive }) =>
    'nav-link' + (isActive ? ' active fw-semibold' : '')
  return (
    <nav className='navbar navbar-dark bg-dark px-3'>
      <span className='navbar-brand'>IT Service Desk</span>
      <div className='d-flex gap-3'>
        <NavLink to='/tickets'      className={linkClass}>Tickets</NavLink>
        <NavLink to='/users'       className={linkClass}>Users</NavLink>
        <NavLink to='/categories'  className={linkClass}>Categories</NavLink>
        <NavLink to='/audit-logs'  className={linkClass}>Audit
Logs</NavLink>
      </div>
    </nav>
  )
}
```

There is no logout button, no user identity display, and no auth-related state in the NavBar. It is a pure presentational component with no local state and no API calls.

8. Pages

The application has five page components, each corresponding to a top-level route. All pages follow the same structural pattern: local state declared with `useState`, a `useEffect` that fetches initial data on mount, form handlers that call the API and then re-fetch the list, and inline error display.

8.1 UsersPage.jsx

UsersPage provides the interface for managing the users who exist in the system. It renders a create-user form and a table of all existing users, each with a delete button. Users are fundamental to the application because tickets, comments, and attachments all reference user IDs.

State Variables

Variable	Type	Initial Value	Purpose
users	Array	[]	List of all users fetched from backend
form	Object	{name, email, password, role}	Controlled form state for create-user
error	String	null	Error message shown in alert if API call fails
loading	Boolean	false	Shows spinner while initial data loads

Create User Form Fields

Field	Input Type	Validation	Maps To
Name	text	required	name
Email	email	required	email
Password	password	required, minLength=8	password
Role	select	required	role (employee support_engineer team_lead admin)

The role select field defaults to 'employee'. On form submission, `createUser(form)` is called, the form is reset to its initial state, and `getUsers()` is called again to refresh the table. On failure, the API error message is stored in the error state and displayed in a Bootstrap danger alert above the form.

Users Table Columns

- Name — full display name of the user
- Email — user's email address
- Role — displayed as a Bootstrap badge; colour varies by role
- Created — formatted creation timestamp
- Delete — button that triggers `window.confirm` then calls `deleteUser(id)` and reloads

8.2 CategoriesPage.jsx

CategoriesPage manages the ticket categories. Categories must exist before tickets can be created, as each ticket requires a `category_id`. The page follows the same create-and-delete pattern as UsersPage.

State Variables

Variable	Initial Value	Purpose
categories	[]	All categories fetched on mount
form	{name, description}	Controlled form state
error	null	Inline error message on API failure
loading	false	Loading indicator

Create Category Form Fields

- Name — text input, required, minimum 2 characters
- Description — text input, optional, free-form text

The categories table shows: Name, Description (an em-dash is rendered if empty), Created date, and a Delete button. Deletion requires `window.confirm` confirmation before calling `deleteCategory(id)`.

8.3 TicketsPage.jsx

TicketsPage is the main working view of the application. It shows the full list of tickets with optional status filtering, provides a form to create new tickets, allows deletion of tickets, and serves as the navigation hub to individual ticket detail views.

State Variables

Variable	Purpose
tickets	Array of ticket objects from the backend
users	Array of users — used to populate the 'Created By' dropdown
categories	Array of categories — used to populate the 'Category' dropdown and resolve category names in the table
statusFilter	Current active status filter string (empty string = show all)
form	Controlled state for the create-ticket form: { title, description, category_id, created_by }
error	Inline error message
loading	Loading indicator for initial data fetch

Status Filter Buttons

A row of Bootstrap buttons allows the user to filter tickets by status. The 'All' button clears the filter and fetches all tickets. Each status button re-fetches tickets with a `?status=<value>` query

string appended. The active filter button is styled differently (btn-primary vs btn-outline-secondary) to indicate the current selection.

Status Value	Badge Colour	Meaning
new	secondary (gray)	Ticket just created, not yet assigned
assigned	primary (blue)	Assigned to a technician, work not started
in_progress	info (cyan)	Technician actively working on the ticket
on_hold	warning (yellow)	Work paused, waiting on information or action
resolved	success (green)	Issue resolved, pending user confirmation
closed	dark (black)	Ticket permanently closed — terminal state

Create Ticket Form Fields

Field	Input Type	Validation	Notes
Title	text	required, minLength=3	Short summary of the issue
Description	text	required, minLength=5	Detailed description of the issue
Category	select	required	Populated from getCategories()
Created By	select	required	Populated from getUsers(); maps to created_by user UUID

Tickets Table

Each row displays: Title (clickable — navigates to /tickets/:id), Status (coloured Bootstrap badge), Category (resolved from the categories array by matching category_id), Created date, and a Delete button. The Delete button calls e.stopPropagation() to prevent the row-click navigation from also firing when the user clicks Delete.

8.4 TicketDetailPage.jsx

TicketDetailPage is the most complex component in the application. It acts as a self-contained action hub for a single ticket, combining all sub-resources (comments, attachments, audit logs) and all available actions (assign, change status, add/delete comments and attachments) into one unified view.

URL Parameter

The page uses useParams() to extract ticketId from the URL path /tickets/:ticketId. This UUID string is passed to all API calls. If the initial ticket fetch fails (e.g., the ticket was deleted), useNavigate() is called to redirect the user back to /tickets.

State Variables

Variable	Purpose
ticket	The single ticket object fetched from GET /tickets/:id
users	All users — used to resolve UUIDs to names throughout the page

comments	Array of comment objects for this ticket
attachments	Array of attachment objects for this ticket
auditLogs	Array of audit log entries scoped to this ticket
error	Page-level error message
assignTo / assignBy	Controlled state for the assign-technician form
newStatus / changedBy	Controlled state for the change-status form
commentContent / commentAuthor	Controlled state for the add-comment form
attForm	Controlled state for the add-attachment form: { filename, url, size, uploaded_by }

Layout Sections

The page is divided into Bootstrap card sections arranged in a responsive grid:

- Back button and error alert — always at the top
- Ticket header card (full width) — shows title, description, status badge, timestamps, and resolved assigned-to name
- Assign Technician card (col-md-6) — two dropdowns: Assign To and Assigned By
- Change Status card (col-md-6) — dropdown of valid next statuses + Changed By dropdown
- Comments card (col-md-6) — add-comment form + list of existing comments with delete buttons
- Attachments card (col-md-6) — add-attachment form + list of existing attachments with links and delete buttons
- Audit Log card (full width) — table of this ticket's audit events

Status Transition Rules

The frontend enforces the same state-machine transitions as the backend. The `NEXT_STATUSES` constant maps each current status to the array of valid next statuses. The Change Status form only shows the statuses that are valid from the ticket's current state. When a ticket is closed, the card displays 'No further transitions available' instead of a form.

```
const NEXT_STATUSES = {
  new:      ['assigned'],
  assigned: ['in_progress'],
  in_progress: ['on_hold', 'resolved'],
  on_hold:  ['in_progress'],
  resolved: ['closed'],
  closed:   [],
}
```

The withError Higher-Order Function

All action handlers (assign, change status, add comment, delete comment, add attachment, delete attachment) are wrapped by a local helper function called `withError()`. This function clears the current error, executes the async action, reloads all page data on success, or sets the error

message on failure. This pattern avoids repeating the same try/catch/reload boilerplate in every handler.

```
const withError = async (fn) => {
  setError(null)
  try {
    await fn()
    await loadAll() // re-fetches ticket, comments, attachments,
auditLogs, users
  } catch (e) {
    setError(e.message)
  }
}
```

API Functions Used

TicketDetailPage is the heaviest consumer of the API client, using 10 of the 20 exported functions:

- `getTicket`, `getUsers` — ticket header data and user name resolution
- `assignTicket`, `updateTicketStatus` — ticket workflow actions
- `getComments`, `createComment`, `deleteComment` — comment management
- `getAttachments`, `createAttachment`, `deleteAttachment` — attachment management
- `getTicketAuditLogs` — ticket-scoped audit trail

8.5 AuditLogsPage.jsx

AuditLogsPage is the simplest page in the application. It is entirely read-only: no forms, no create or delete controls. It displays a table of every audit log entry across all tickets in the system, ordered newest-first as returned by the backend.

State Variables

- `logs` — array of all audit log objects fetched on mount
- `loading` — boolean loading indicator

Table Columns

Column	Display Logic
Action	Displayed as a Bootstrap info (cyan) badge
Ticket ID	First 8 characters of the ticket UUID followed by '...' to save space
Performed By	First 8 characters of the performer UUID followed by '...' — NOT resolved to a name
Details	Raw details string from the audit log record
When	Formatted using <code>toLocaleString()</code> for localised date/time display

Note: unlike TicketDetailPage's audit log section (which resolves performer UUIDs to user names using the loaded users array), AuditLogsPage does NOT fetch the users list and therefore cannot resolve UUIDs to names. The Performed By column shows a truncated UUID. This is a known limitation documented in Section 13.

9. Application Flow & User Journey

9.1 Required Data Creation Order

The data model has foreign-key relationships that must be satisfied. Users and categories must exist before tickets can be created. Therefore, the recommended setup order for a fresh system is:

- Step 1 — Navigate to /users and create at least one user. Users are referenced as ticket creators, ticket assignees, comment authors, attachment uploaders, and status-change actors.
- Step 2 — Navigate to /categories and create at least one category. All tickets require a category.
- Step 3 — Navigate to /tickets and create tickets, selecting the user and category created above.
- Step 4 — Click a ticket row to open TicketDetailPage and perform workflow actions.

9.2 Ticket Lifecycle Flow

A ticket's journey through the system follows a strict state machine. The frontend enforces these transitions via the NEXT_STATUSES map, and the backend validates them independently. The full lifecycle is:

- Created with status: new
- Assigned to a technician → status becomes: assigned
- Technician starts work → status becomes: in_progress
- Work paused → status becomes: on_hold (can return to in_progress)
- Issue resolved → status becomes: resolved
- Ticket closed → status becomes: closed (terminal — no further transitions)

9.3 Navigation Paths

All navigation between pages is handled by React Router. The primary paths are:

From	Action	Navigates To
Any page	Click nav link	Corresponding route
Browser / NavBar	Visit /	/tickets (automatic redirect)
TicketsPage	Click ticket row	/tickets/:ticketId
TicketDetailPage	Click Back button	/tickets
TicketDetailPage	Ticket not found (error)	/tickets (programmatic navigate)

9.4 Data Refresh Strategy

The frontend uses a simple optimistic-consistency strategy: every mutation (create, delete, assign, status change, etc.) is followed by a complete re-fetch of all data relevant to the current page.

There is no local cache, no optimistic update, and no partial update of the local state array. While this approach increases the number of API calls, it guarantees that the UI always reflects the true backend state and eliminates an entire class of stale-data bugs.

10. Data Models

The following data models represent the shape of JSON objects returned by the FastAPI backend and consumed by the frontend. All IDs are UUID strings generated by the backend. The frontend never generates IDs.

10.1 User

Field	Type	Notes
id	UUID string	Backend-generated primary key
name	string	Display name
email	string	Unique email address
role	string enum	employee support_engineer team_lead admin
created_at	ISO 8601 str	Timestamp of account creation

10.2 Category

Field	Type	Notes
id	UUID string	Backend-generated primary key
name	string	Category display name (min 2 chars)
description	string null	Optional description
created_at	ISO 8601 str	Timestamp of category creation

10.3 Ticket

Field	Type	Notes
id	UUID string	Backend-generated primary key
title	string	Short issue summary (min 3 chars)
description	string	Detailed issue description (min 5 chars)
category_id	UUID string	Foreign key → Category.id
status	string enum	new assigned in_progress on_hold resolved closed
created_by	UUID string	Foreign key → User.id — the reporter
assigned_to	UUID null	Foreign key → User.id — the assignee, null if unassigned
created_at	ISO 8601	Creation timestamp
updated_at	ISO 8601	Last modification timestamp

10.4 Comment

Field	Type	Notes
id	UUID string	Primary key
ticket_id	UUID string	Parent ticket reference
author_id	UUID string	Foreign key → User.id
content	string	Comment text body
created_at	ISO 8601	Timestamp

10.5 Attachment

Field	Type	Notes
id	UUID string	Primary key
ticket_id	UUID string	Parent ticket reference
uploaded_by	UUID string	Foreign key → User.id
filename	string	Display name of the file
url	string	URL where the attachment can be retrieved
size	integer	File size in bytes
created_at	ISO 8601	Upload timestamp

10.6 Audit Log

Field	Type	Notes
id	UUID string	Primary key
ticket_id	UUID string	The ticket this event relates to
action	string enum	created assigned status_changed
performed_by	UUID string	Foreign key → User.id — actor of the event
details	string	Human-readable description of the event
created_at	ISO 8601	When the event occurred

11. State Management Architecture

The frontend deliberately avoids any global state management solution. There is no Redux store, no Zustand atom, no React Context provider, and no custom hooks. Every page component is entirely self-contained: it declares its own state with `useState`, fetches its own data with `useEffect`, and handles its own mutations.

11.1 The `useState` + Fetch-on-Mount Pattern

Every page in the application follows this identical pattern:

- Declare state with `useState` for the data arrays, form fields, loading flag, and error message
- `useEffect` with an empty dependency array (`[]`) runs once on component mount
- The effect calls the relevant API function(s) and populates state via `setState`
- Form submission handlers call the API, reset the form, and call the load function again
- Delete handlers call `window.confirm`, then call the delete API, then reload
- Errors are caught in `try/catch` blocks and stored in the error state variable
- The error state is displayed as a Bootstrap danger alert in the JSX

11.2 Why No Global State?

The decision to use only local component state is appropriate given the application's scope. No data needs to be shared between pages simultaneously — each page is a distinct view that is fully unmounted when the user navigates away. Because data is always fetched fresh from the backend on page mount, there is no stale-data problem to solve, and no need to synchronise a client-side cache with the server.

A global store would add complexity (actions, reducers, selectors) without providing any benefit at this scale. If the application grows to require real-time updates, cross-page notifications, or optimistic UI, Context API or a lightweight store like Zustand would be straightforward additions.

11.3 No Custom Hooks

The codebase contains no custom React hooks (no `useFetch`, `useTickets`, etc.). Each page component directly calls `useEffect` and `useState`. While custom hooks could reduce some boilerplate, the current approach keeps every page's data logic entirely visible in one file, making it easy to understand and modify each page in isolation.

12. Backend Integration

12.1 The Vite Development Proxy

During development, all API calls are made to `http://localhost:5173/api/*` — the same origin as the frontend. Vite intercepts any request whose path begins with `/api`, strips the `/api` prefix, and forwards the request to `http://localhost:8000/*`. This means the browser never makes a cross-origin request during development, and no CORS configuration is needed on the FastAPI backend for local development.

In production, the equivalent behaviour must be configured at the infrastructure level: either an nginx location block that proxies `/api/` to the backend, or a cloud load balancer rule that routes based on URL prefix.

12.2 Complete API Endpoint Reference

Method	Path	Called From	Purpose
GET	<code>/users</code>	UsersPage, TicketsPage, TicketDetailPage	List all users
POST	<code>/users</code>	UsersPage	Create a new user
DELETE	<code>/users/{id}</code>	UsersPage	Delete a user by ID
GET	<code>/categories</code>	CategoriesPage, TicketsPage	List all categories
POST	<code>/categories</code>	CategoriesPage	Create a category
DELETE	<code>/categories/{id}</code>	CategoriesPage	Delete a category
GET	<code>/tickets</code>	TicketsPage	List tickets (optional <code>?status=</code> filter)
GET	<code>/tickets/{id}</code>	TicketDetailPage	Fetch single ticket
POST	<code>/tickets</code>	TicketsPage	Create a ticket
DELETE	<code>/tickets/{id}</code>	TicketsPage	Delete a ticket
PATCH	<code>/tickets/{id}/assign</code>	TicketDetailPage	Assign technician
PATCH	<code>/tickets/{id}/status</code>	TicketDetailPage	Change ticket status
GET	<code>/tickets/{id}/comments</code>	TicketDetailPage	List comments for ticket
POST	<code>/tickets/{id}/comments</code>	TicketDetailPage	Add a comment
DELETE	<code>/tickets/{id}/comments/{cid}</code>	TicketDetailPage	Delete a comment
GET	<code>/tickets/{id}/attachments</code>	TicketDetailPage	List attachments
POST	<code>/tickets/{id}/attachments</code>	TicketDetailPage	Add an attachment
DELETE	<code>/tickets/{id}/attachments/{aid}</code>	TicketDetailPage	Delete an attachment
GET	<code>/audit-logs</code>	AuditLogsPage	All system audit logs
GET	<code>/tickets/{id}/audit-logs</code>	TicketDetailPage	Ticket-scoped audit logs

12.3 Error Handling Contract

The frontend expects that when the backend returns a non-2xx HTTP status code, the response body contains a JSON object with a 'detail' field containing a human-readable error message. This is the default FastAPI error response format. The API client's `request()` function reads this field and uses it as the Error message that is ultimately displayed to the user in the inline error alert.

13. Limitations & Future Improvements

The current implementation represents a functional Minimum Viable Product (MVP) for an IT service desk frontend. The following items are intentionally out of scope for the initial version and represent natural candidates for future development sprints.

13.1 Authentication & Authorisation

The frontend has no authentication layer. There is no login page, no JWT token management, no auth headers on API requests, and no route guards. Any user with network access to the frontend can view and modify all data. Adding authentication would involve: a `/login` page with a credential form, token storage (likely `localStorage` or `httpOnly` cookies), an `Authorization` header injected by the API client, and `ProtectedRoute` wrappers that redirect unauthenticated users.

13.2 No Edit Forms

The frontend supports creating and deleting records, but not editing them. There are no edit forms for users, categories, or ticket metadata. The backend exposes `PUT` endpoints that the frontend does not call. Adding edit functionality would require pre-populated forms, additional API client functions wrapping the `PUT` endpoints, and inline edit mode or modal dialogs in each page's table.

13.3 No Pagination

All list endpoints are fetched in full with no pagination query parameters. For small datasets this is acceptable, but for a production system with thousands of tickets or audit log entries, this will cause performance problems and slow page loads. Pagination would require `page/offset` state in each page component, `Previous/Next` navigation controls, and corresponding query parameters in the API client.

13.4 No TypeScript

The codebase uses plain JavaScript (JSX). TypeScript would provide compile-time type checking, improved IDE autocompletion, and self-documenting function signatures — particularly valuable for the data models in Section 10 and the API client's function signatures. Migrating to TypeScript would involve adding `tsconfig.json`, renaming `.jsx` files to `.tsx`, and adding type definitions for all data models.

13.5 No Test Suite

There are no unit tests, integration tests, or end-to-end tests. Adding Vitest (the natural choice for a Vite-based project) with React Testing Library would enable component-level testing. Playwright or Cypress would cover end-to-end workflows. The simplicity of the codebase — pure functions, direct API calls, minimal abstraction — makes it well-suited for testing.

13.6 Audit Log UUID Resolution

AuditLogsPage displays truncated UUIDs in the Performed By column rather than resolved user names. TicketDetailPage solves this by loading the users array and looking up names by UUID. AuditLogsPage could do the same by adding a getUsers() call on mount.

13.7 No Real-Time Updates

Ticket status changes made by other users are not reflected until the browser is manually refreshed. Adding WebSocket support or periodic polling (using setInterval in a useEffect) would allow the UI to reflect live changes without requiring page refreshes.

13.8 No .env Configuration

The API base URL ('/api') is hardcoded in client.js. In a more mature project, this would be externalised to an environment variable (import.meta.env.VITE_API_BASE_URL) so that different environments (dev, staging, production) can target different backends without code changes.

14. Running & Developing the Application

14.1 Prerequisites

Requirement	Version	Notes
Node.js	18+	npm is bundled with Node.js
Python	3.11+	Required for the FastAPI backend
MongoDB	6+	Required by the backend; must be running locally
npm	9+	Used to install frontend dependencies and run scripts

14.2 First-Time Setup

Install Frontend Dependencies

```
cd C:\Rakesh\Frontend\frontend
npm install
```

This installs all packages listed in package.json into the node_modules\ directory.

Start the Backend

```
# In a separate terminal, from the backend project root:
uvicorn app.main:app --reload
# Backend listens on http://localhost:8000
```

Start the Frontend Development Server

```
cd C:\Rakesh\Frontend\frontend
npm run dev
# Frontend dev server starts on http://localhost:5173
# Vite proxy: http://localhost:5173/api/* → http://localhost:8000/*
```

Open <http://localhost:5173> in a browser. The application loads and immediately redirects to /tickets.

14.3 Development Workflow

Vite's Hot Module Replacement (HMR) means that saving any .jsx or .js file in the src\ directory will instantly update the browser without a full page reload, preserving component state in most cases. The React Fast Refresh plugin ensures that component state is preserved during module hot-swaps when possible.

Adding a New Page

- Create a new file in frontend\src\pages\ (e.g., ReportsPage.jsx)
- Export a default functional component
- Import it in App.jsx
- Add a <Route> entry in the Routes block
- Add a <NavLink> in NavBar.jsx if the page should appear in navigation

Adding a New API Function

- Open frontend\src\api\client.js
- Add an exported arrow function that calls request() with the appropriate method, path, and optional body
- Import the new function in the page component that needs it

14.4 Production Build

```
cd C:\Rakesh\Frontend\frontend
npm run build
# Output: frontend\dist\
# Serve dist\ with any static file server or CDN
```

The build command runs Vite's Rollup-based bundler, which tree-shakes unused code, minifies JavaScript and CSS, and outputs hashed asset filenames for cache-busting. The resulting dist\ directory can be deployed to any static hosting service (Nginx, AWS S3 + CloudFront, Netlify, Vercel, etc.).

Previewing the Production Build Locally

```
npm run preview
# Serves frontend\dist\ on http://localhost:4173
# NOTE: The Vite proxy does NOT run in preview mode.
#       Configure a separate reverse proxy for the backend.
```

14.5 File Dependency Map

The following diagram shows the full import graph of the application:

```
main.jsx
├─ App.jsx
│   ├── NavBar.jsx          (react-router-dom: NavLink)
│   ├── UsersPage.jsx       (api/client: getUsers, createUser,
deleteUser)
│   ├── CategoriesPage.jsx  (api/client: getCategories,
createCategory, deleteCategory)
│   ├── TicketsPage.jsx     (api/client: getTickets, getUsers,
getCategories,
│                           createTicket, deleteTicket)
│   │                           (react-router-dom: useNavigate)
│   ├── TicketDetailPage.jsx (api/client: getTicket, getUsers,
assignTicket,
│                           updateTicketStatus,
getComments,
│                           createComment, deleteComment,
│                           getAttachments,
createAttachment,
│                           deleteAttachment,
getTicketAuditLogs)
│   └─ AuditLogsPage.jsx    (react-router-dom: useParams, useNavigate)
│                           (api/client: getAllAuditLogs)
api/client.js
├─ fetch (native browser Fetch API – no Axios or other HTTP library)
```